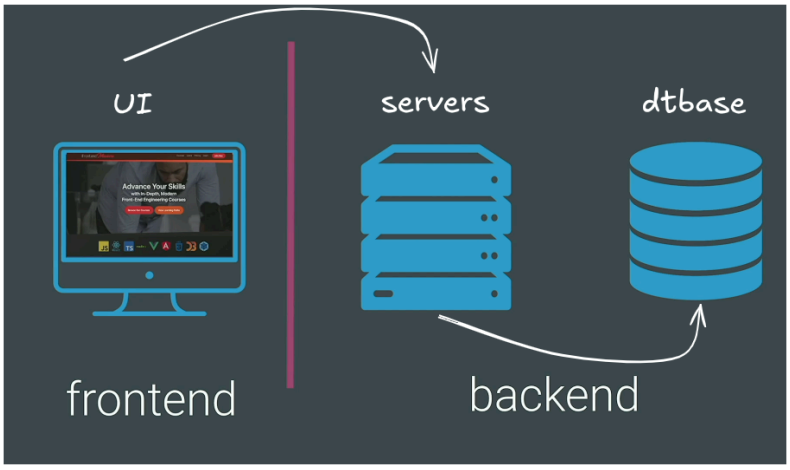
# Full Stack for Front-End Engineers, v3

2026-01-18 10:17

Tags:

---

## Introduction




- Servers and networking
- DTBases overview
- How the internet works
  => And the relationships between them

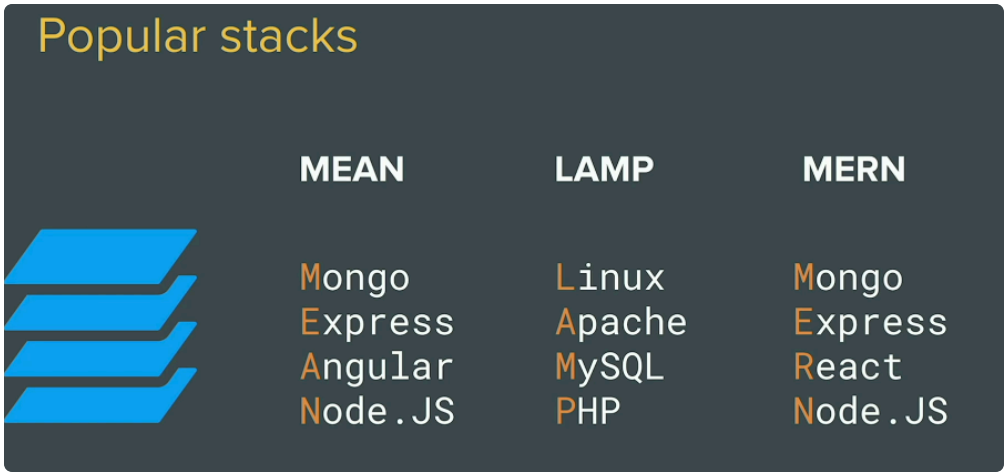
### What makes a Fullstack Engineer?

genaralist?
jack of all trades? (and master of none)
swiss knife of codes? (lol)
knows how they fit together?
=> Front End + Back End

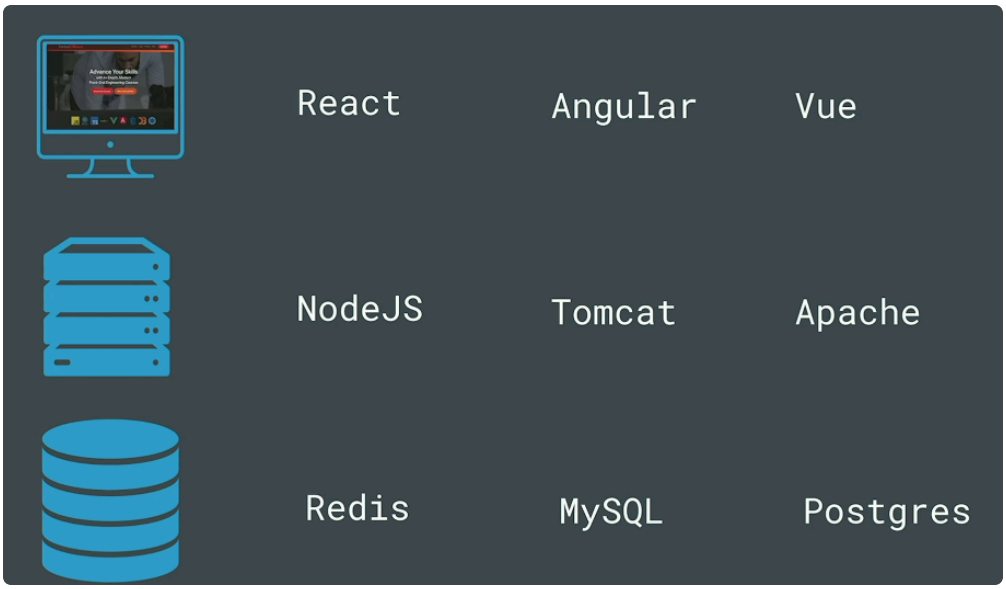
***Stack la cdj?***

- UI
- Web server
- Databases
- operating system

- application server (potentially)




Tên thế thôi nma marketing cả -.-, có nhiều stack vl




Kqtrong lắm, chỉ cần thấy stack nào mình thấy thoải mái và put things together, hiểu đc nó là đc

# Diving in

- termianls and command line
- VIM
- Shells

***Terminal & Command line***

aka cách nói chuyện với máy tính, ra lệnh etc...

- fast
- consistent
- low overhead

```
ls - list directory contents
cd - change directory
mkdir - make directory
rmdir - remove directory


cat - show file contents
man - command manual
less - show file contents by page
touch - creates an empty file
rm - remove file
echo - repeat input
```

rmdir phế vcl vì nó k xóa được dir nếu có content bên trong
-> dùng rm -rf (-r : recursively)

Trong trường hợp lỗi

```
cd .. - move up one directory
pwd - print working directory
clear - clear terminal output
ctrl + c - exit program
cd ~  - navigate to home directory
```

pwd: print current working dir

mkdir -p (extend the "make", including the subdir)

## VIM

Có các mode:

- insert mode (editiing): ***i***
- normal mode (primary mode): ***ESC***
- command mode (search / save / exit): ***:***
  save and exit ***:wq***
  undo: ***ESC + U*** (or U in command mode)

## Shell

bash: bourne again shell (?)
./zshrc : file config của shell zsh
./bash_profile: của bash

source <shell config location>: reload lại shell với file config

echo $SHELL để biết đang dùng shell gì
-> dùng echo để output ra text trên terminal

Biết thêm để customize shells [Introduction to Bash, VIM & Regex with Substack (James Halliday) | Frontend Masters]

# Server

- what is a server
- cloud computing
- VPS
- operating systems
- ssh

Server để làm gì?

- serve content + respond to requests
- using specialized hardware (for handling large amount of request)
- 100% availability (to other devices)
- chạy server trên lap cá nhân có bị làm sao không? (thì gập laptop server sập chứ sao -.-)

*Basic node server*

```
const http = require("http");
const fs = require('fs'); // file system
const PORT = 3000;

const server = http.createServer(function (req, res) { // server nao cung nhan vao req: request and res: respond
    res.writeHead(200, { 'content-type': 'text/html'}); // status code: 200 - successful response
    fs.createReadStream('index.html').pipe(res); // index.html chưa tạo nên để đó thou, với stream thì chỉ cần head của file thôi xong nó tự chảy (không cần static từng cái nữa (?))
});

server.listen(PORT); // server chưa biết làm gì cả, nên muốn nó listen cái port
consolde.log('Server da bat dau o cong ${PORT}'); // log ra để xem nó có đang chạy hay k
```

-> Cách chạy đoạn code trên

1. Tải node.js (nếu dùng Arch -> dùng yay để tải)
   nvm để quản lí phiên bản node
2. Chạy bằng lệnh `node <ten file>.js`

```
1   const http = require('http');Û
  1 const fs = require('fs');
  2 const PORT = 3000;
  3
  4 const server = http.createServer(function (req, res) {
  5     res.writeHead(200, {'content-type': 'text/html'});
  6     fs.createReadStream('index.html').pipe(res);
  7 });
  8
  9 server.listen(PORT);
 10 console.log(`server started on port ${PORT}`);
```

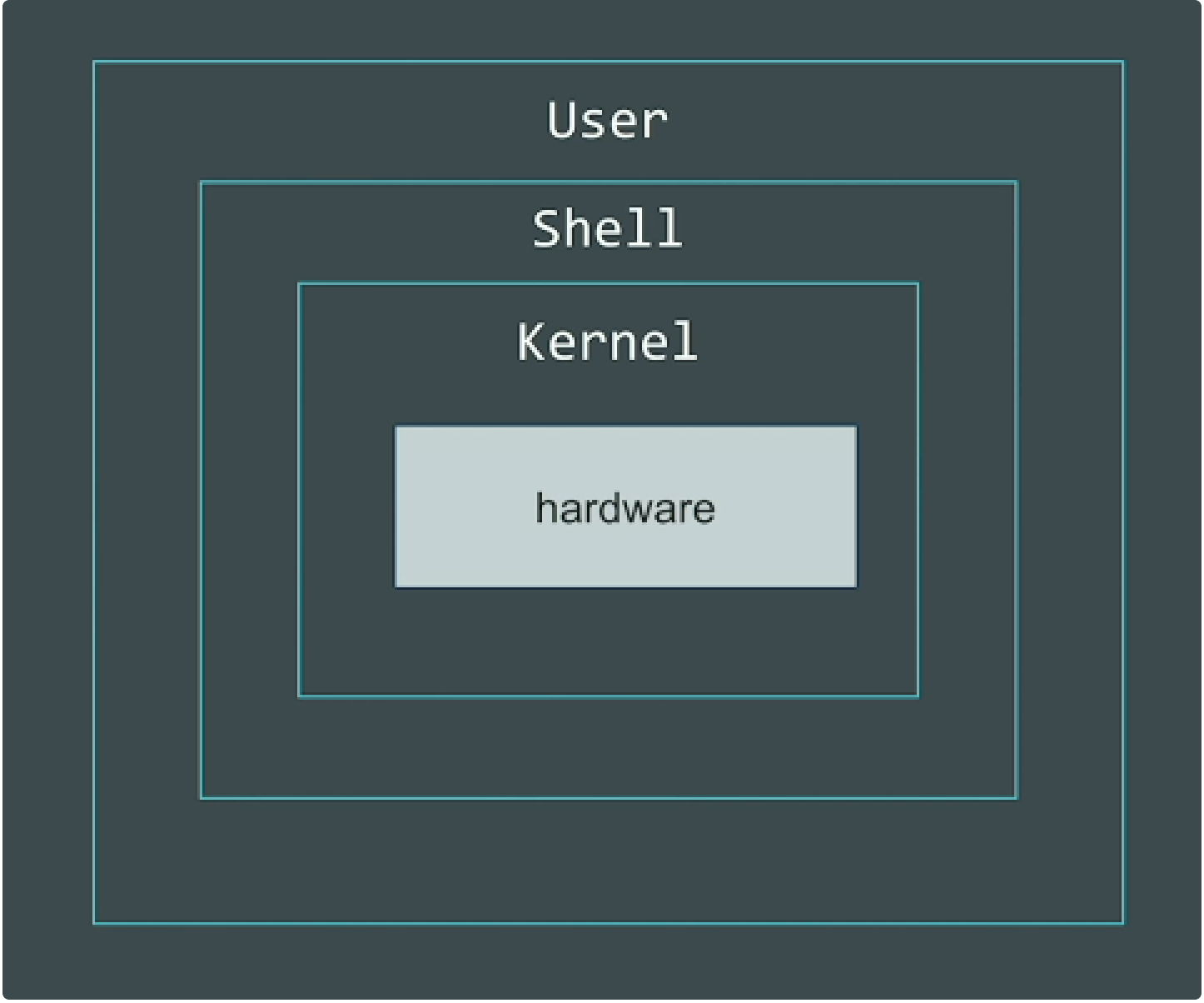
   Lưu ý, phải dùng dấu ` (backtick) để in template
3. Viết file index.html để chạy

***Cloud computing***

- virtualization: dividing a servers resources into virtual computers
- virtual machine: digital version of a physical computer
- VPS: virtual private server

đã mua, khuyến khích đừng đki của DO tốn vcut :( dki cái free của Azure để học

# Operating system




Security

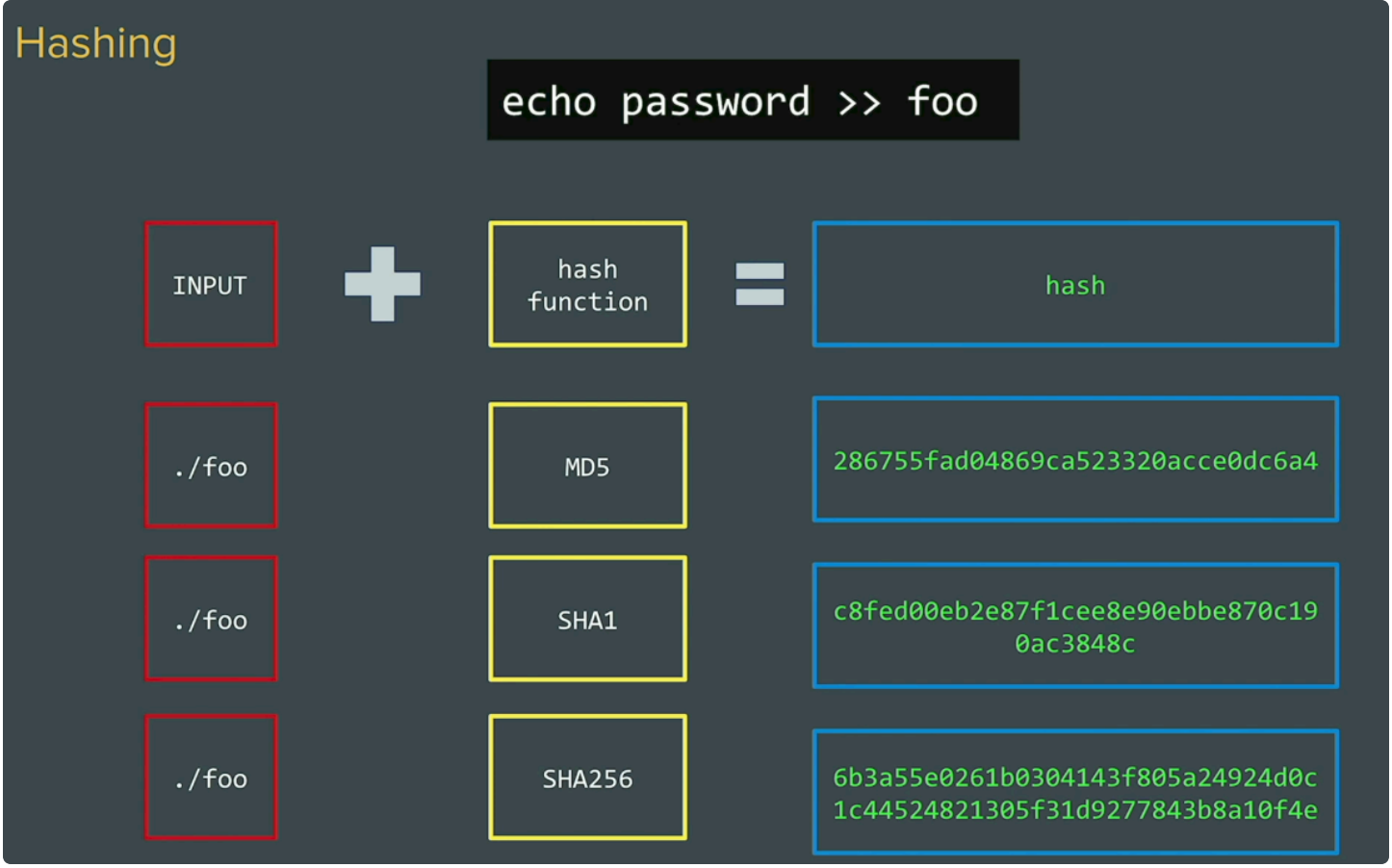
Làm sao để truy cập được server

- username / password (not reliable cho lắm, ai có đc bộ này đc đều truy cập đc)
- biometric
- ssh key (gen ra bởi máy tính, not human) 💫

> professional tips (dùng 1Password, đừng dùng LastPass aka cái t đang dùng :) )

🔥 **Dictionary attack**

Hashing

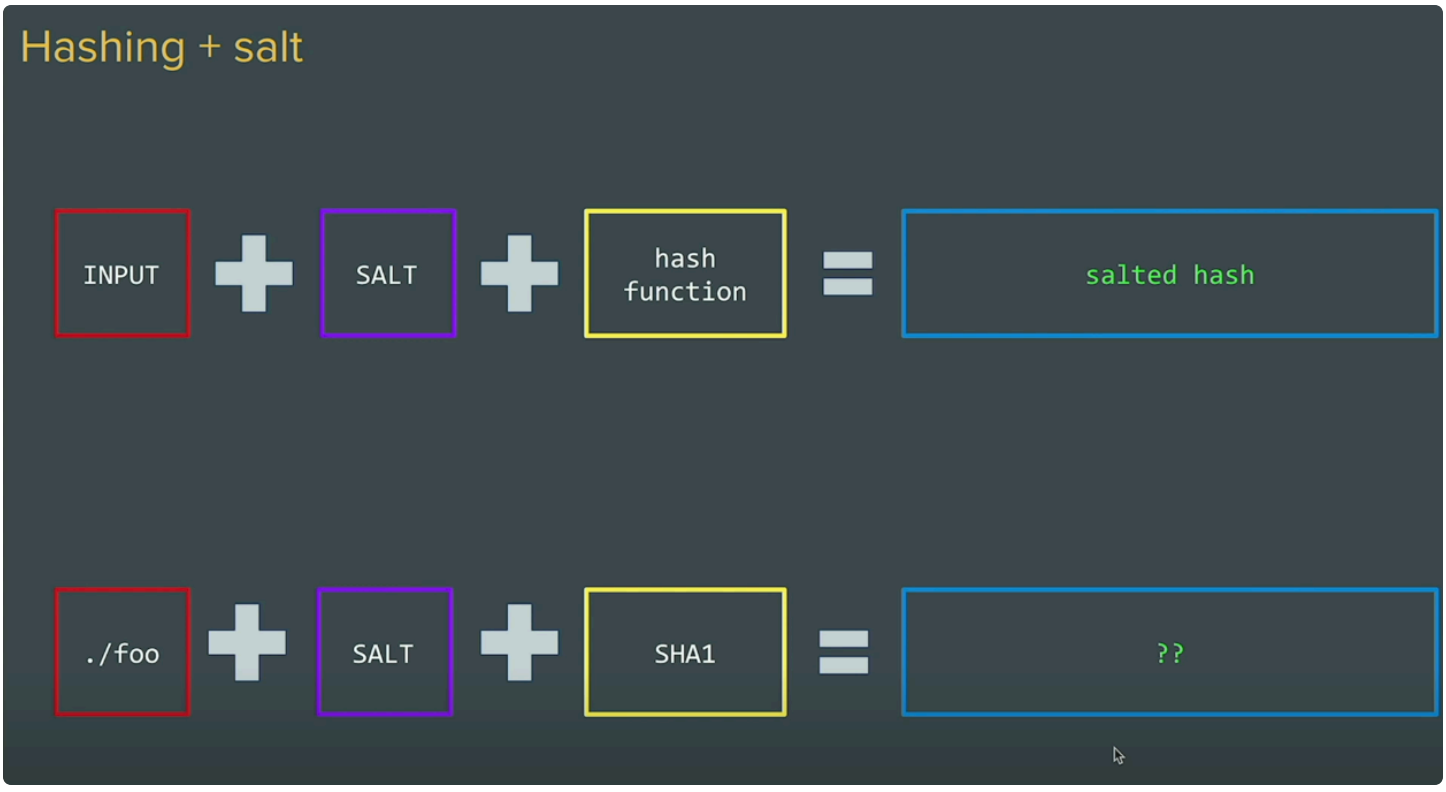



> Rainbow table

Sử dụng `openssl` để access các công cụ này

```
openssl <type of hash> <file for hashing>
```

A SALTED HASH???



SALT is randomness
Thay vì hash chỉ riêng phần password, ta thêm tí "muối" - SALT mà nó thật là ngẫu nhiên, đặc trưng cho cái quá trình đó => Mặc dù hacker biết thuật toán rồi nhưng không biết SALT của mình

SALT có thể là bất cứ thứ gì

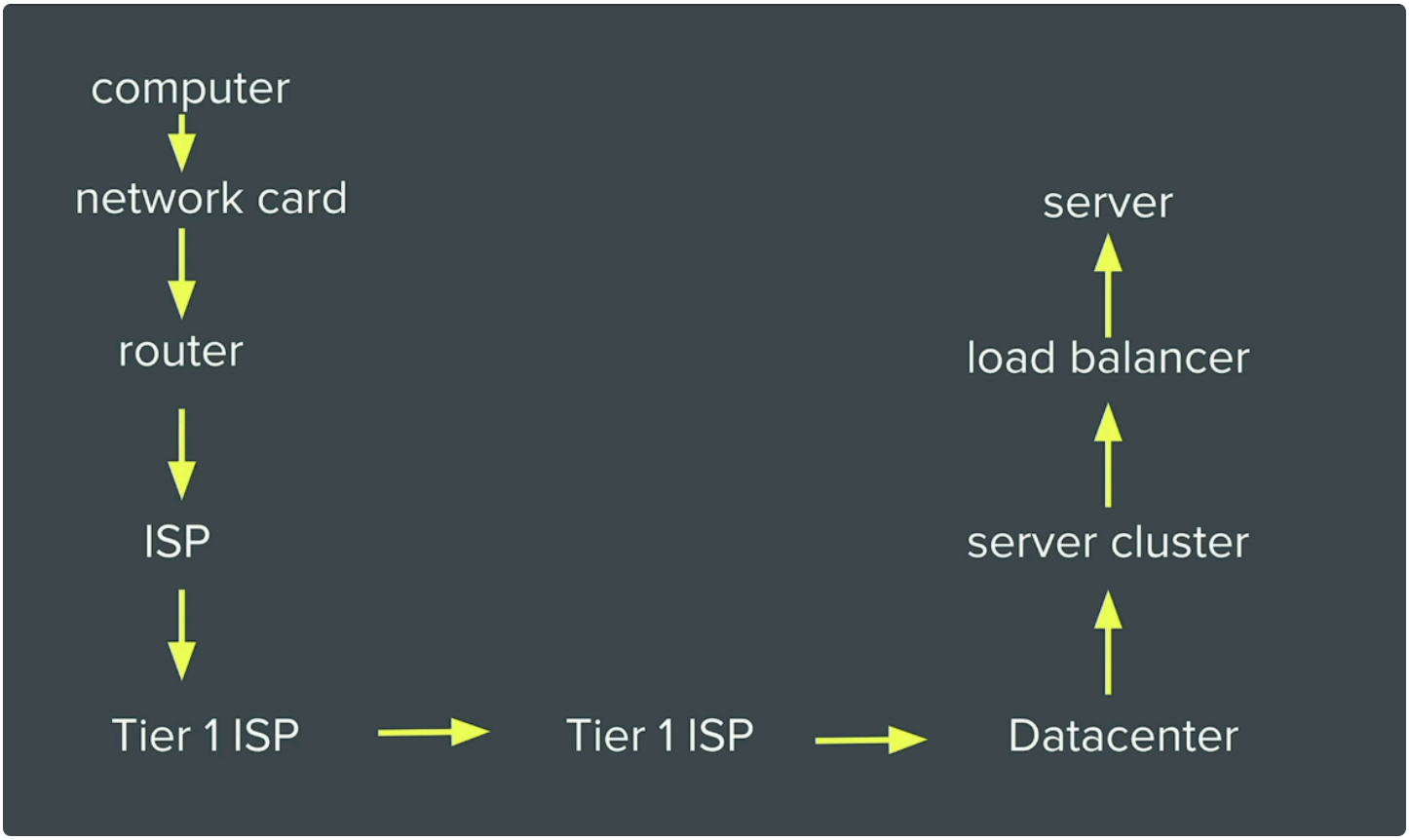
Vốn dĩ các hàm hash đều là các hàm toán học -> Ai cũng biết cái hàm này -> tránh đc reverse engineering

ssh - Secure Socket Shell
bssh key pair:

- public key: chia sẻ public
- private key: mình có private có thể decrypt được và chỉ mình decrypt được

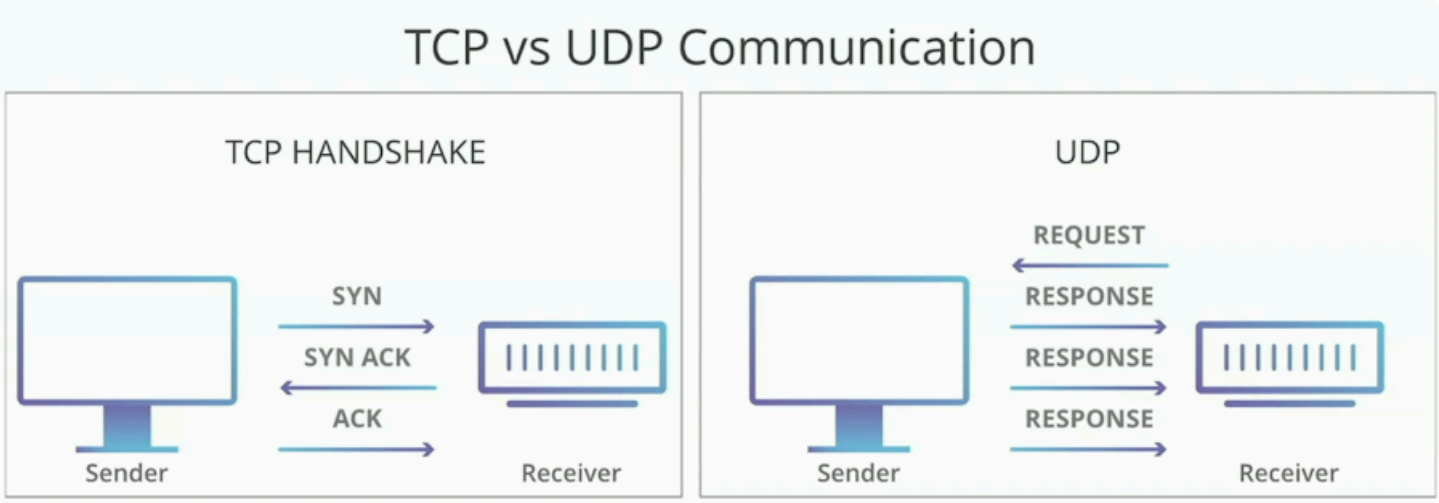
# Understanding the internet

Internet is built on cooperation and rules

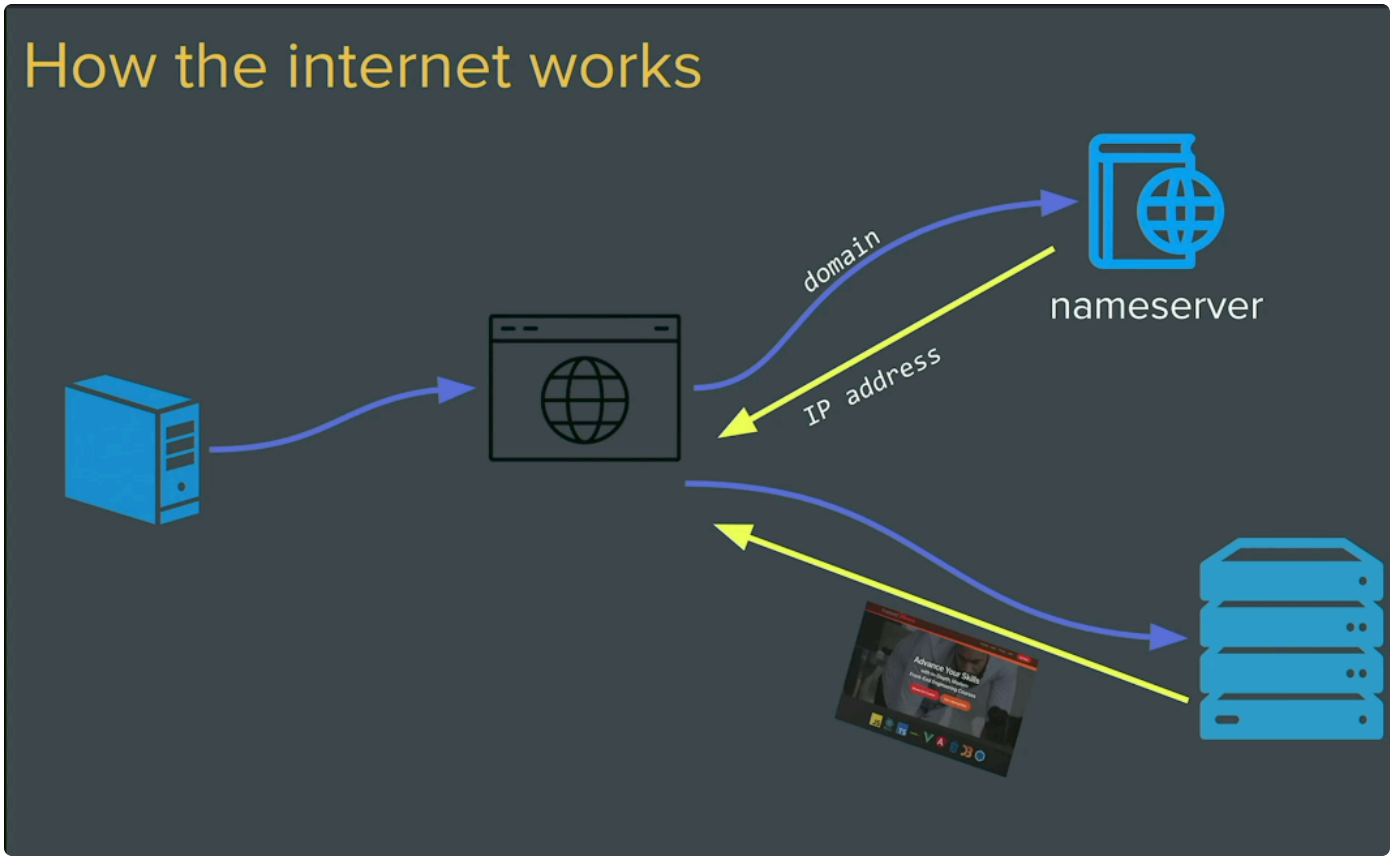



Terminology
ISP: Nhà cung cấp mạng?
Internet: a network of networks

intranet: private network
IP: internet protocol

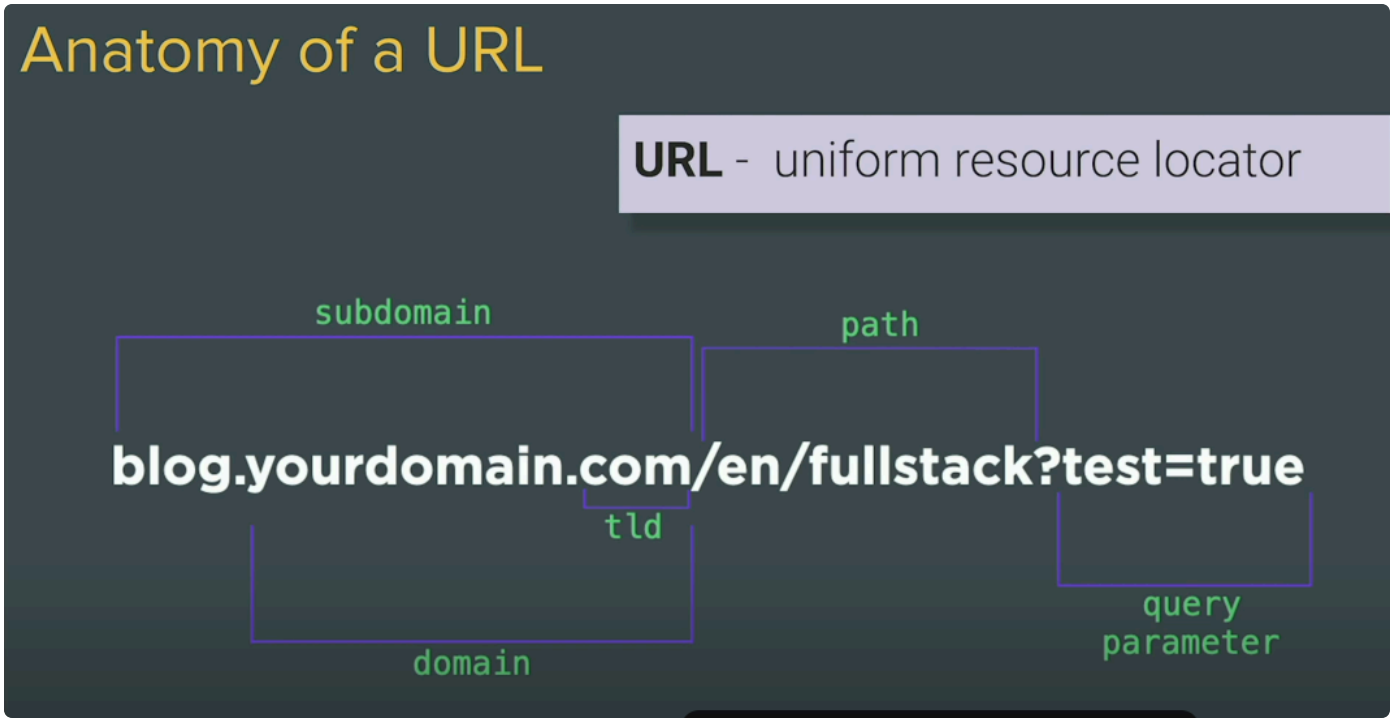



- TCP - *Transmission Control Protocol*: Một protocol để giao tiếp mạng. Kiểu bro tôi gửi file cho ông (server), ông có nhận được không thì bảo tôi (thiếu chỗ nào để tôi gửi lại (ACK stands for acknowledge)). *Chậm hơn nhưng reliable*
  (HANDSHAKE là điểm khác biệt - aka có sự giao tiếp)
- UDP - *User Datagram Protocol*: Kiểu tao ném này, mày bắt nhé, ném tùm lum kiểu gì mà chả hết ~~ (thường dùng trong streaming videos, các thứ cần phải nhanh, nhiều frame, realtime)
- ICMP: internet control message protocol. Chir check server, kiểu điều hành check xem mọi thứ còn sống k (ping of death - ping cho đến chết, server k thể nhận thêm request nữa (or DDOs))
- Packet: unit of data transmitted over a network
- DNS: domain name system, cái chuyên map các ip với các domain




- DNS records:

    - A record: maps name to IP
    - CNAME: maps name to name
- URL: uniform resource locator




Network tools

`ping google.com`: check status của network host
`traceroute google.com`: track đường đi của request (all network hops between front to end point)
`netstat` : show network status

- tất cả trạng thái hiện tại của mạng, ip, ports ...
  `nslookup`: tra cứu từ domain ra ip
  `dig`: tra cứu DNS rec cho một domain

# Server Setup

- server setup
- web servers
- nodeJS

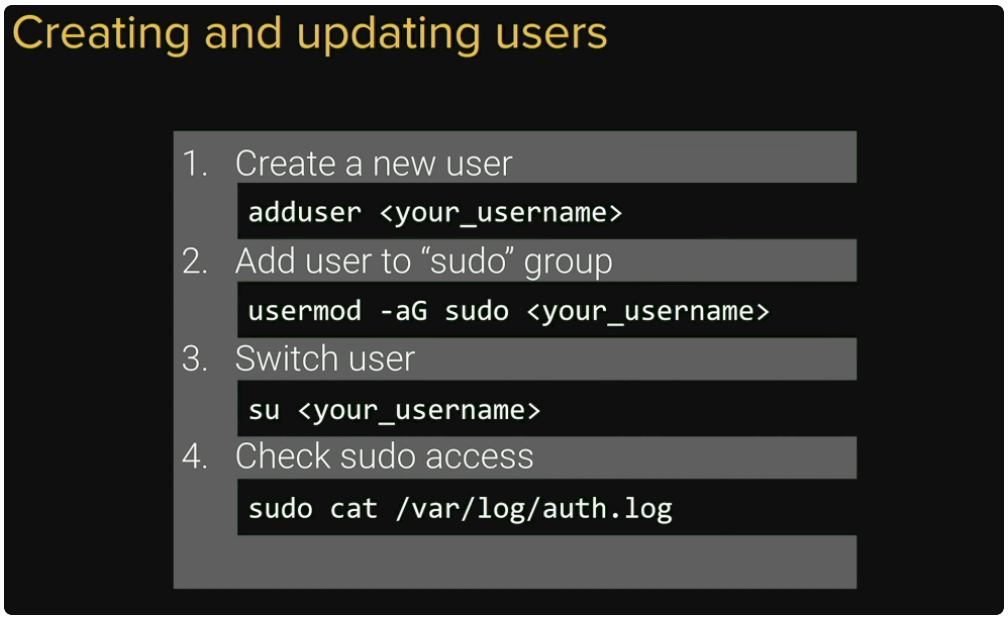
Server setup

1. Update software
    1. ssh to the server
    2. update software
       `apt update`
       `apt upgrade`

2. restart the server

   `shutdown now -r` // flag for restart

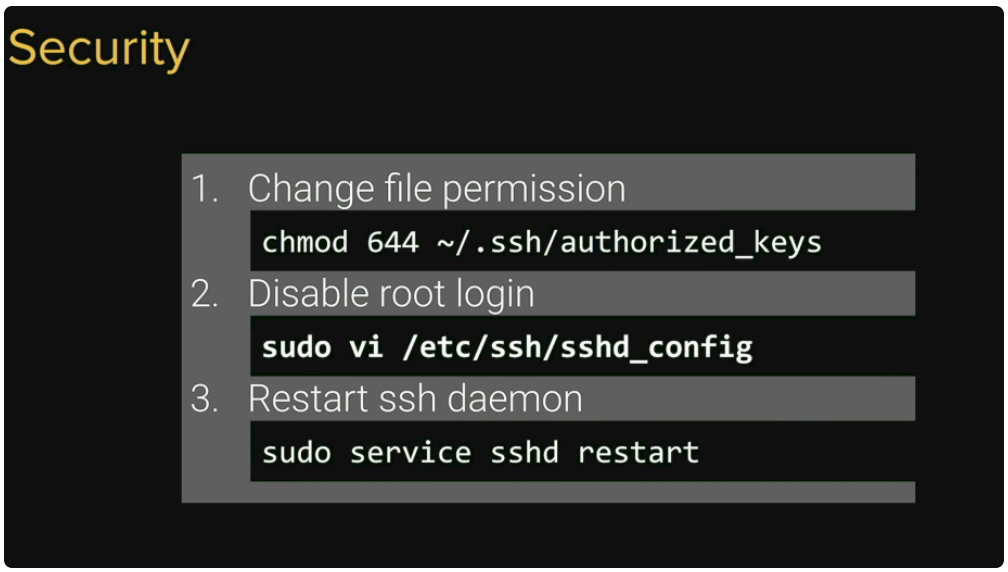
3. create a new user




4. make that user a SUPERUSER
5. enable login for new user

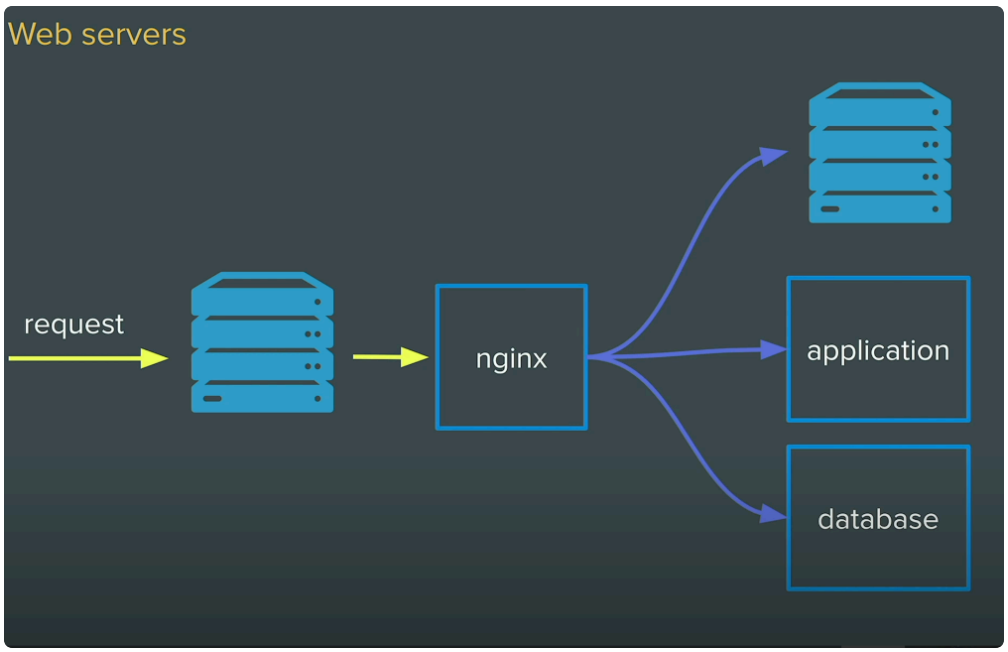



6. disable root login

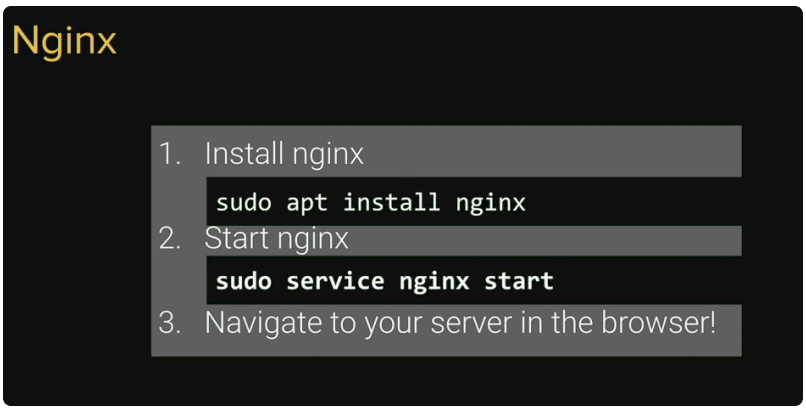


Web Server

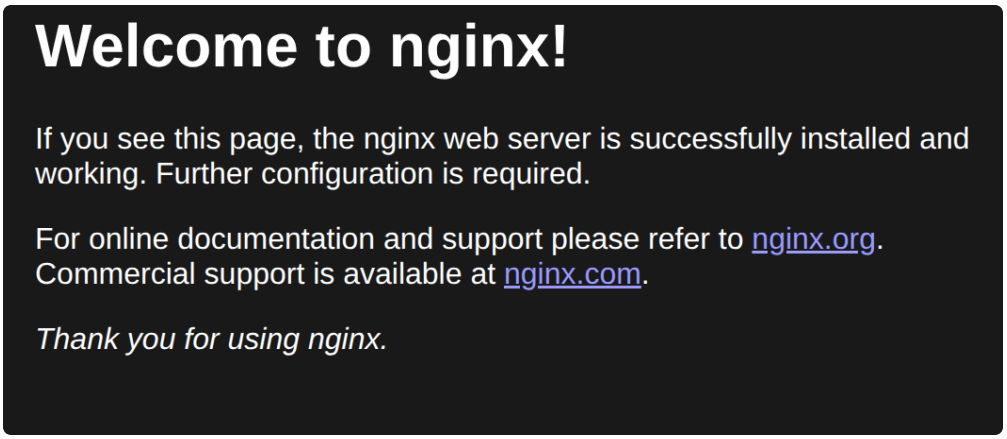
- Apache
- nginx



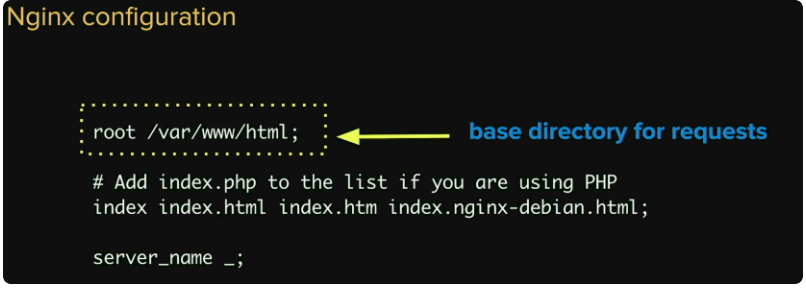

nó đóng vai trò như người điều hướng, xem request muốn đi về đâu thì lấy cái đó cho request



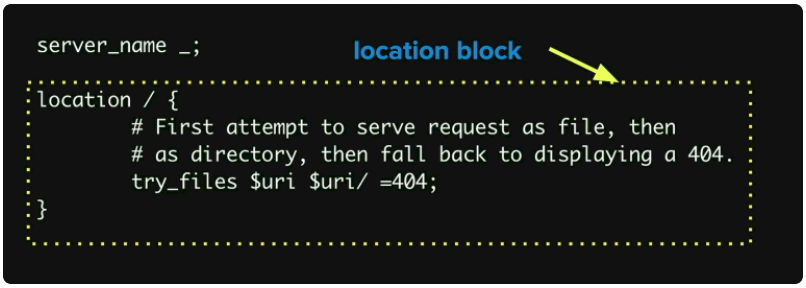

Đến đoạn này thử gõ trên browser nó ra cái này




Và nội dung của html mặc định trên được chứa ở dưới này, mình có thể mô đi phê cái này để view cái file html của mình



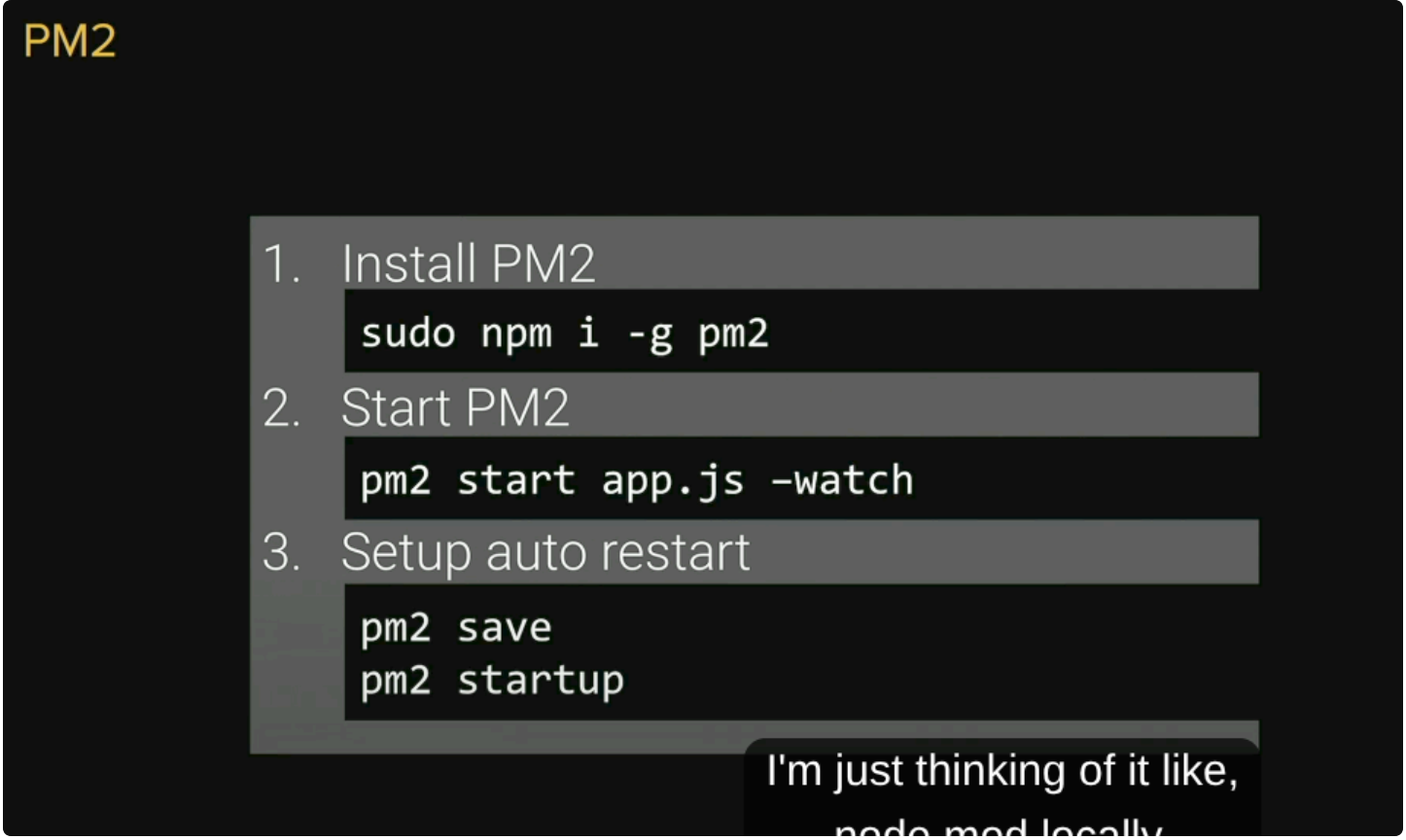

Cái này để điều hướng trên URL




## Application setup

về khái niệm proxy và virtual server:
kiểu là với mỗi server, ví dụ đang chạy trên
rimorocus.id.vn (đang chạy trên Digital Ocean)
mình muốn stack thêm 1 server nữa chồng lên, để xử lí các tác vụ khác nhau thì thêm dấu / ?
rimorocus.id.vn/ xxx / xxx





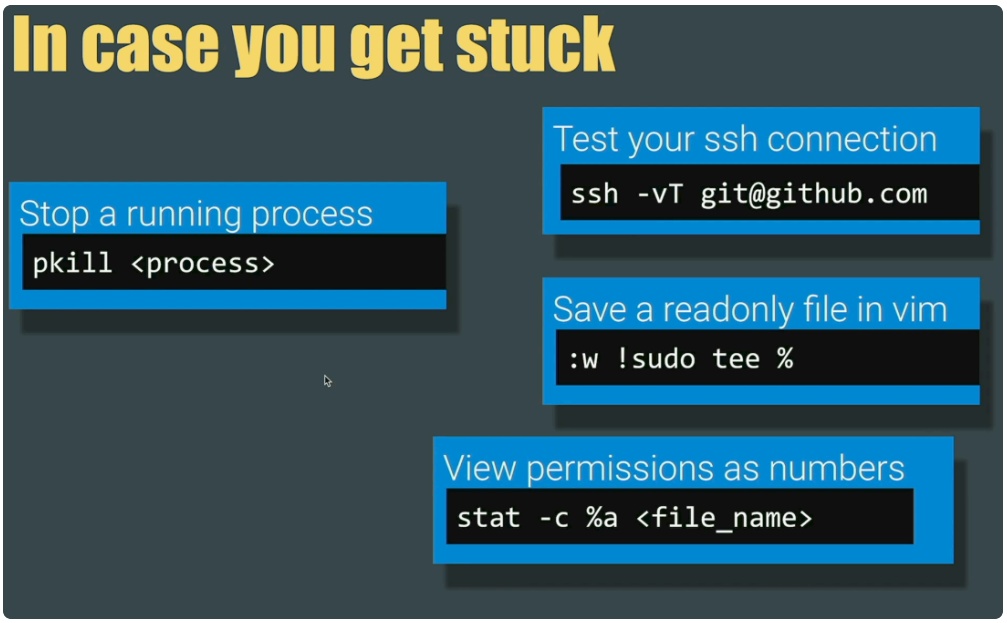

## Version control with Git




1. Làm trên web
2. Dùng `ssh-keygen`
3. Add trên web luôn
4. `git remote add origin git@github.com:DatHoang-git/fullstack-to-frontend.learn.git`
5. 

`git add . && git commit -m 'First commit'`

> *Quản lý dự án ở* `/var/www/app`



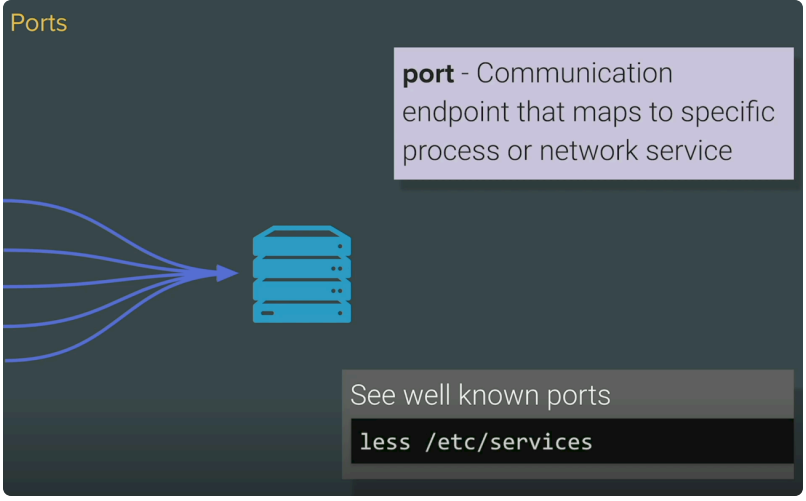

# I MIGHT NEED SECURITY

1. Acess control
2. Securing applications
3. Firewalls
4. Permissions
5. Updating software

Cải thiện Security

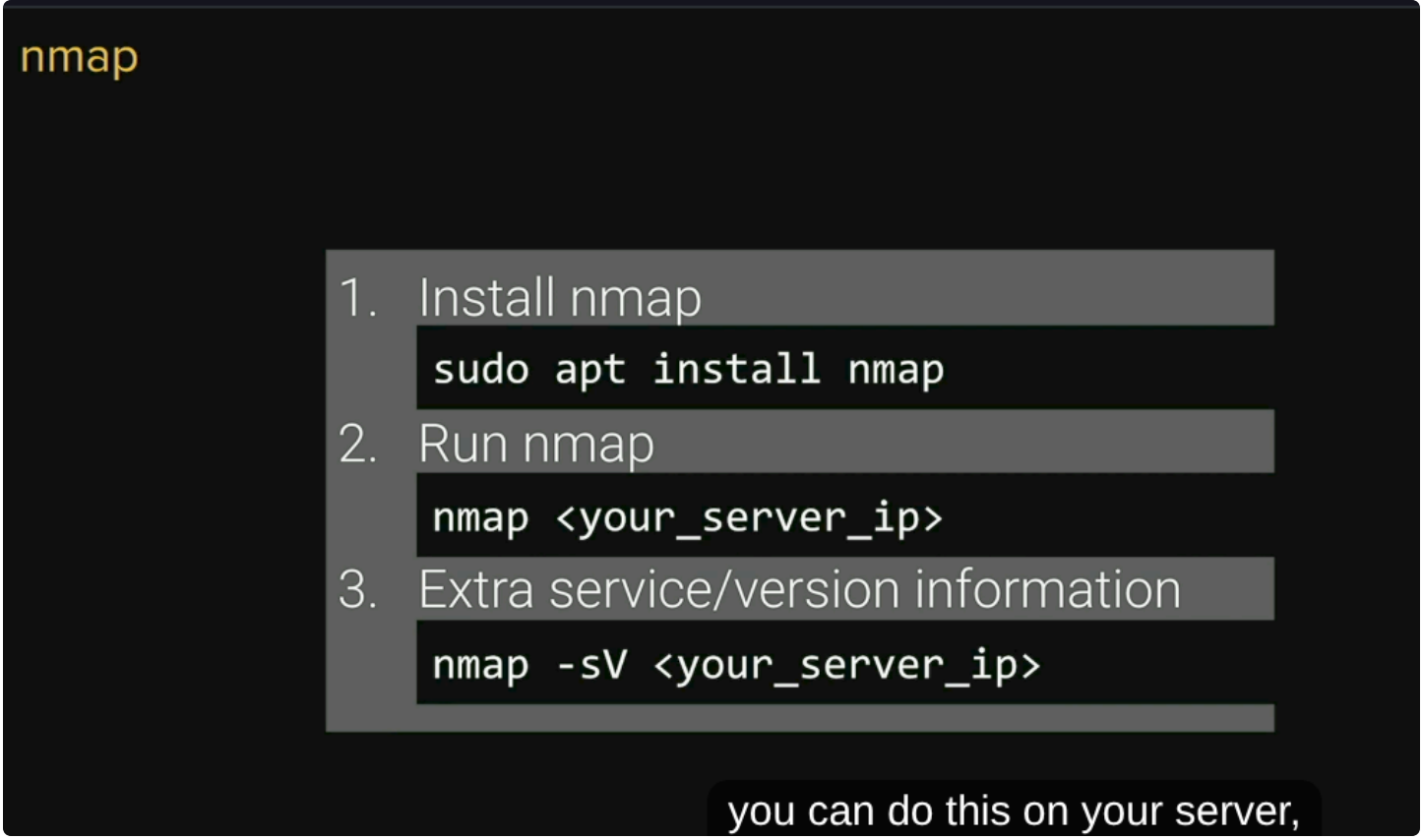
- ssh
- firewalls
- updates
- two factor authentication
- vpn

## About Ports


Ports
**port** - Communication endpoint that maps to specific process or network service
See well known ports
less /etc/services




Tại sao lại dùng 8000 hay 8080 (Để tránh colision)




nmap - Network Maper:
Dùng để quét mạng và kiểm tra bảo mật:

- Gõ từng port xem cái nào đang mở
- Hỏi xem service nào đang ở trong
- Xem máy đó chạy gì

Luôn mở các app sau:
22 - port ssh
80/tcp - port http

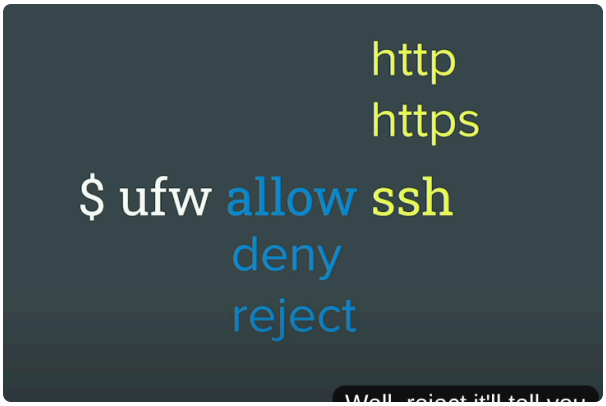
443 - https

Firewall

- A network security **device**
- monitors **incoming and outgoing** network
- **decides** whether to allow or block
- **based on rules** (difined set of security rules)

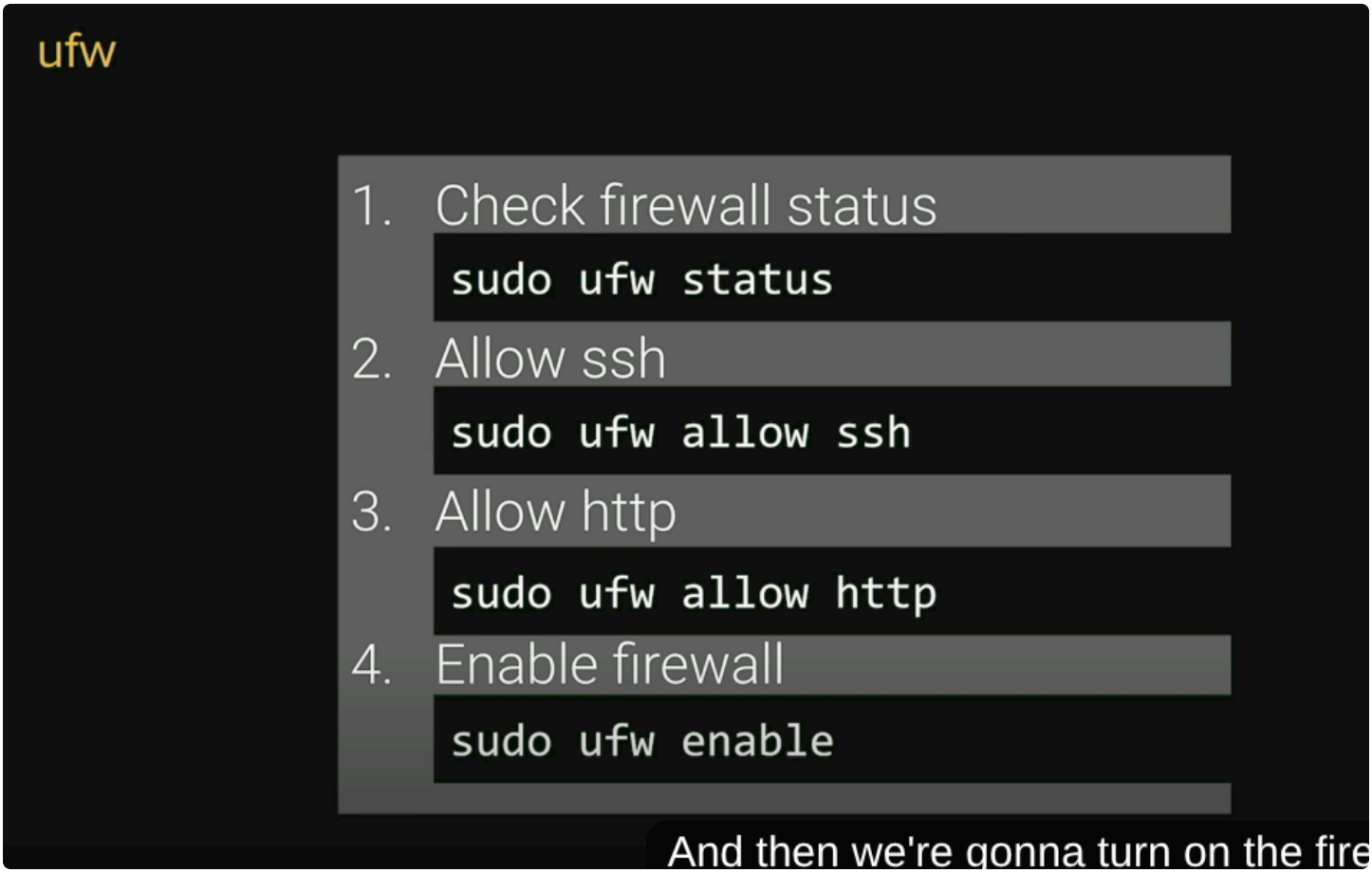
```
ufw - uncomplicated firewall
```

```
!= ip table
```




deny khác với reject thế nào??

- deny: phủ nhận / từ chối quyền truy cập *deny sends a blocking noti*
- reject: bác bỏ / loại bỏ (chặn ngay và luôn, thực sự là sẽ không hiện gì) *reject is silent*



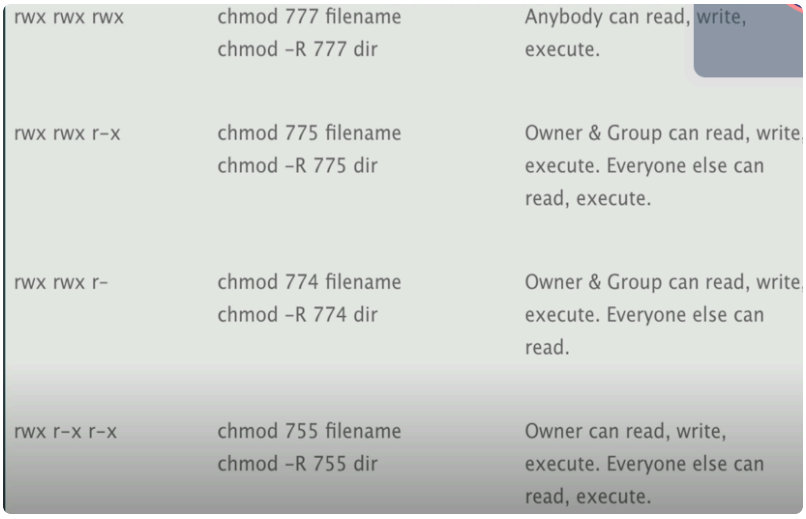

Tại sao cho phép https thay vì http?

Permissions

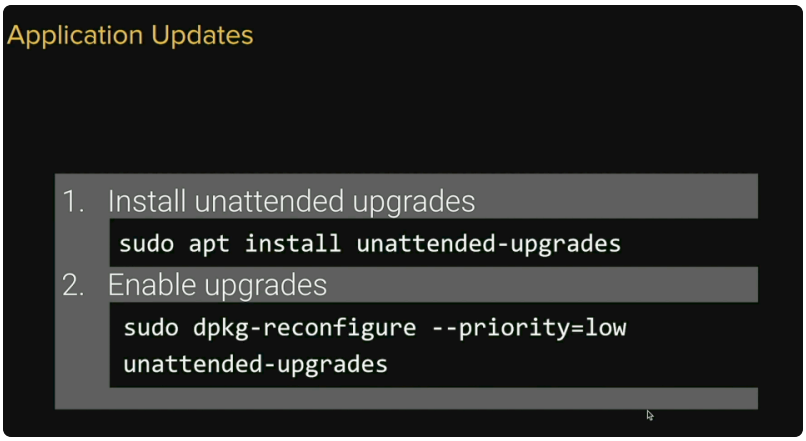



$2^0$: execute
$2^1$: write
$2^2$: read
-> dùng base 8 - octo

rwx: 4 + 2 + 1 = 7 (cho tất các quyền)
dùng theo cú pháp:
owner - group - everyone else
7 7 7 (tất cả đều có rwx)




## Application updates



- Tự động cập nhật, không cần update, patching thủ công
- Nhưng tại sao nên để update ở priority thấp?
    - Vì mình muốn tối ưu tài nguyên cho web server hơn là việc cập nhật -> Tránh bị lag

# Diving into the server pt2

- CI/CD
- Shell scripting
- Cron

### CI/CD

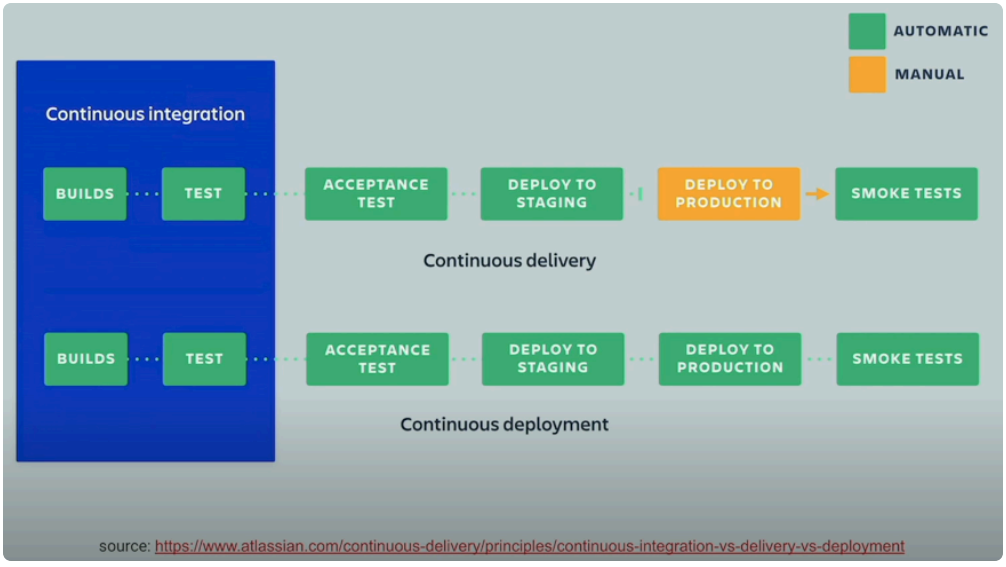
CI means: ***Continuous Intergration***
Sự thay đổi trong code được xác nhận (validated, check var xem chuẩn chưa (!! quan trọng đấy)) và merge về branch chính càng thường xuyên / càng sớm càng tốt

- Mục đích: Phát hiện lỗi được viết ra, tránh tích tụ

CD means: ***Continuous Delivery & Continuous Deployment***
Sự thay đổi code được tự động built và sẵn sàng để deploy trên production (cũng tự động nốt)

- Đảm bảo phần mềm luôn ở trạng thái có thể phát hành
- CD(elivery) khác nhau với CD(eployment) như thế nào?:
  CDeli là những thay đổi đã sẵn sàng nhưng **không được tự động push lên prod**
  CDeploy là mọi thứ tự động từ A-Z

Là một tiêu chí / triết lý trong việc triển khai dự án. Ta không muốn ào một phát push hết lên xong bug tùm lum, ta ưu tiên các sự thay đổi nhỏ có thể resonable, thì dễ tìm ra bug hơn



TESTING / CODE VALIDATION IS THE CORE OF CI/CD

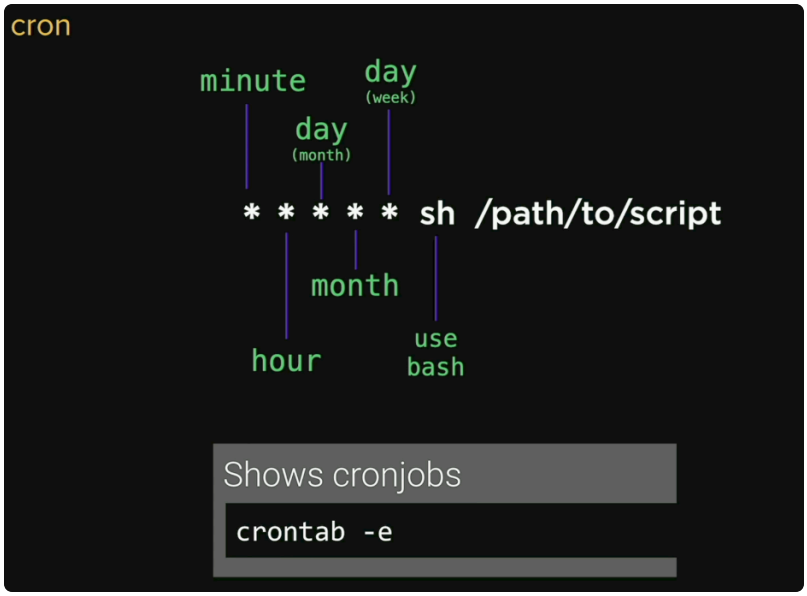
Spinnaker: tìm hiểu sau, khá overkill cho dự án cá nhân, sau này các cty lớn có thể dùng, để tạo CI/CD pipeline

=> Mục đích của CI/CD: TỐC ĐỘ / RELIABLENESS
***Github Actions*** (bạn Hà kì trước dùng):

- Push code lên Github
- Nếu test xanh, phần mềm tự động đẩy lên production, pull code về và restart

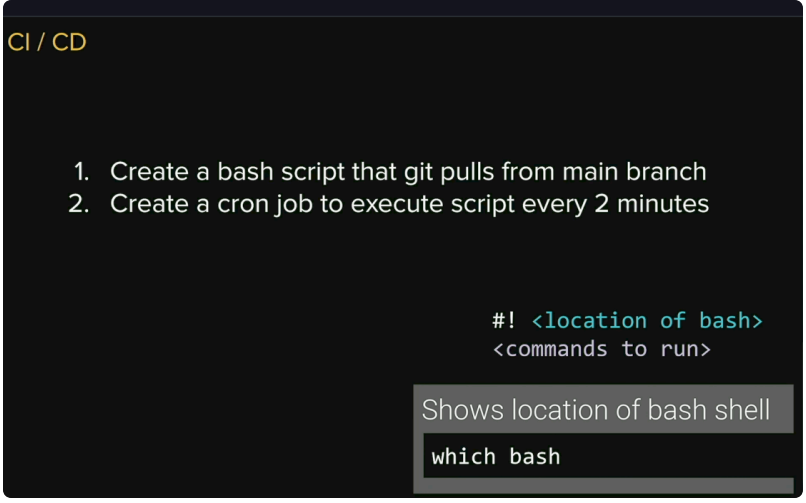
cron `cron - executes commands on a schedule`

- Như một công cụ (timer) để set intervals cho các câu lệnh / scripts -> Nền tảng để viết CI/CD pipeline




Cấu hình như nào, tham khảo Crontab.guru - The cron schedule expression generator, dễ đọc expression / syntax hơn

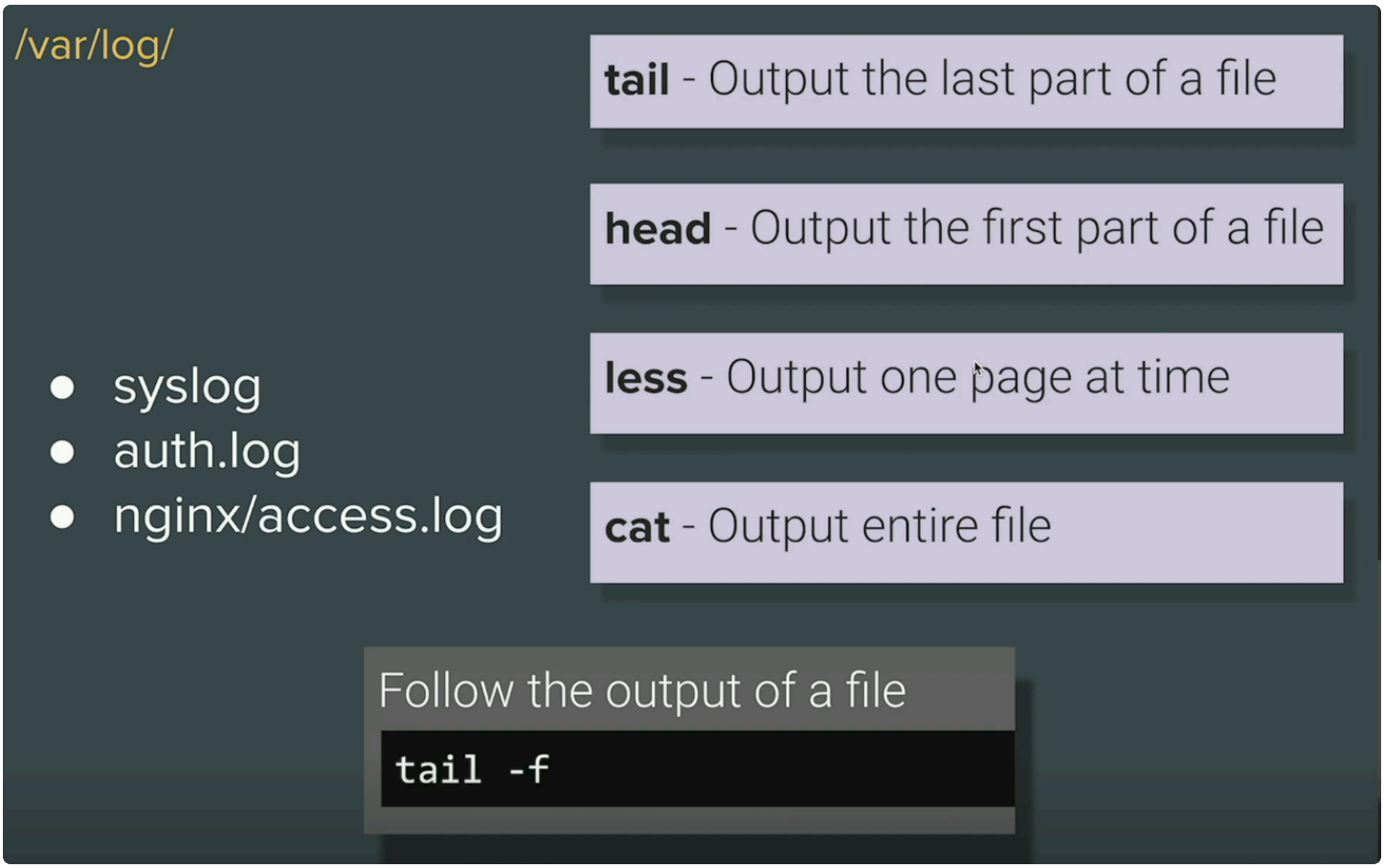
Shell scripting
Làm shell script (.sh)
Exercise:




-> Tự động edit, thoải mái edit trên vscode, cái gì cũng được mà không cần phải thủ công cài lên VPS từ github

## Master the terminal

- Logs and how to read them
- Standard streams
- Redirection
- Finding things




Các log đều chứa ở (và nên chứa ở)

```
/var/log/
```

Để xem file log realtime thì dùng

```
tail -f
```

## Standard Streams

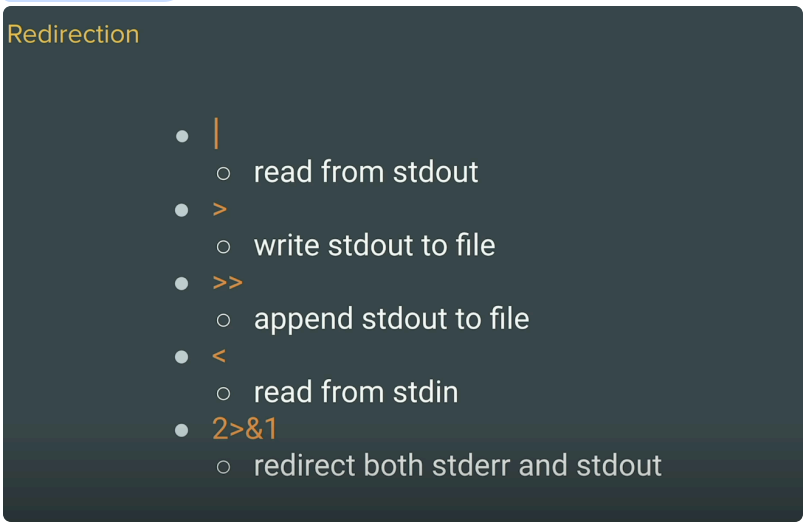
- standard output: stdout
- stdin
- stderr (error)

-> Trong hệ UNIX, với tất cả các command ta chạy (stdin), có output (stdout), và cả err (sdterr)
=> Mọi command chỉ lấy 3 arguments này thôi, it ***fits for all*** command, mặc xác nó khác nhau như thế nào (siêu siêu tiện luôn)

## POSIX - ***Portable Operating System Interface***

Cái này đặc trưng ở UNIX, là bộ tiêu chuẩn để đảm bảo tính tương thích giữa họ UNIX

## Redirection




```
> : Có override lên file viết ra

>> : Append - nối vào file đã có

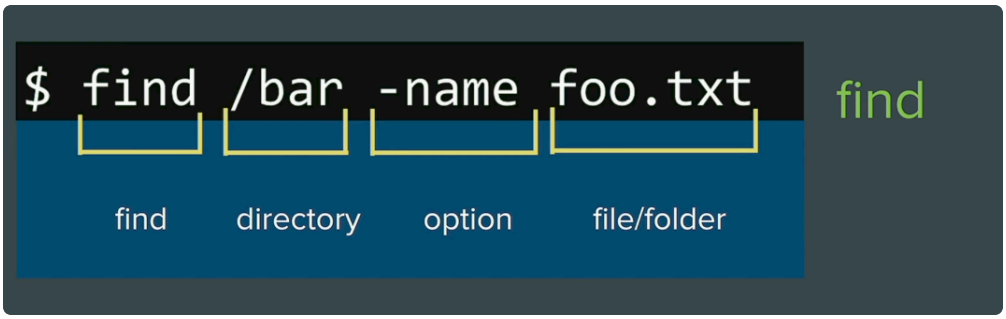
2>&1 : Có gì từ output thì in ra hết, cả output cả err
```

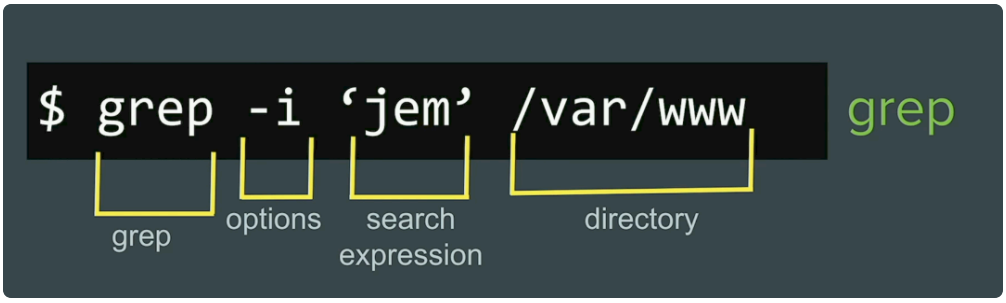
Khác với pipeline trong KTMT, pipeline ở đây nối đuôi nhau về mặt dữ liệu (data xử lí ở command (output) là input của command sau)

Finding things - Tìm kiếm các thứ trong system của mình

```
find : search file names

grep : search file contents
```




> use `zgrep` for looking in zip files

# Nginx

- nginx redirection
- Gzip
- Subdomains

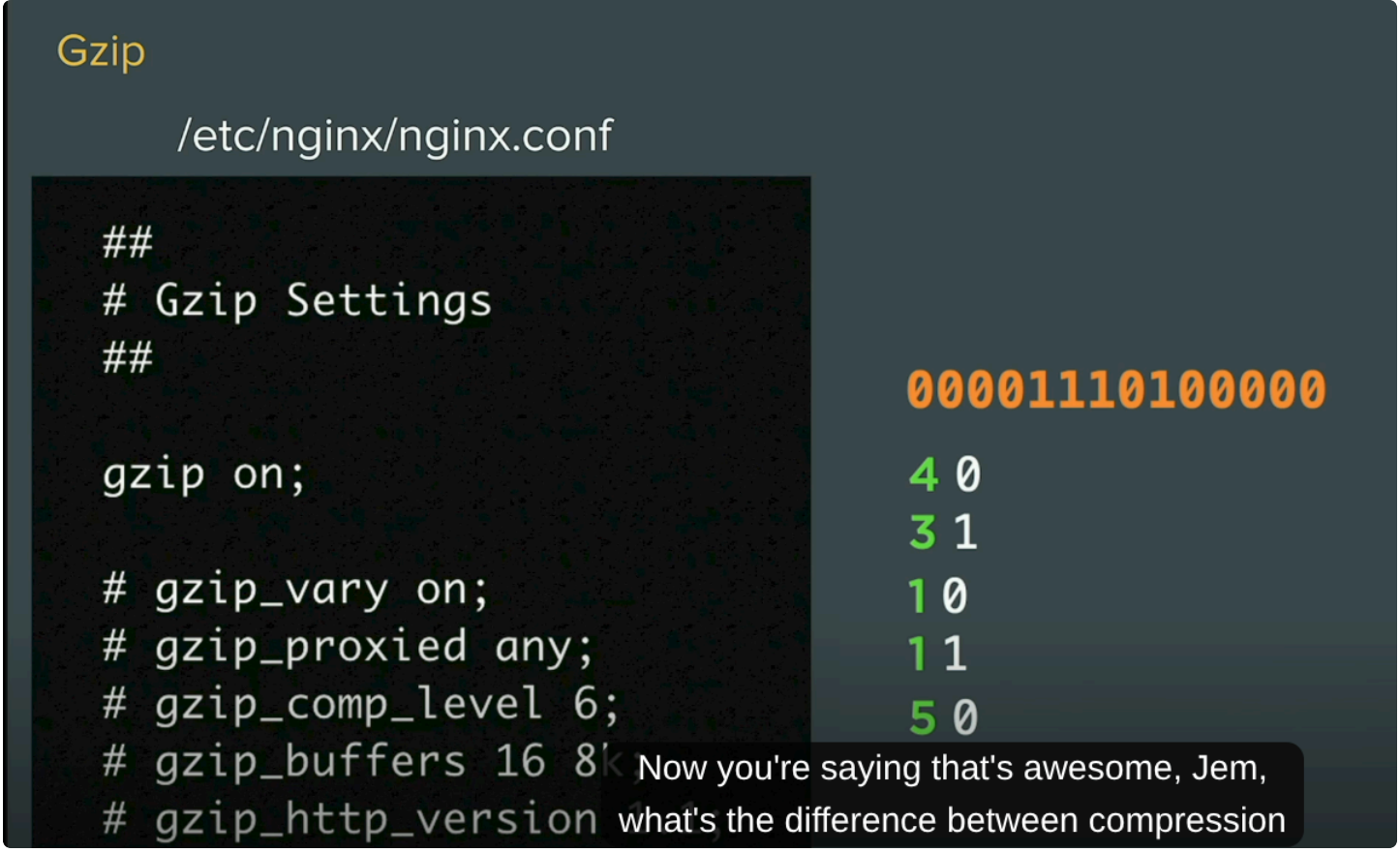
Gzip

Compression algo running on nginx

Why compression then?

Nén fiele lại để giảm kích cỡ truyền tin -> file truyền nhanh hơn và hiệu quả hơn

How does compression works?

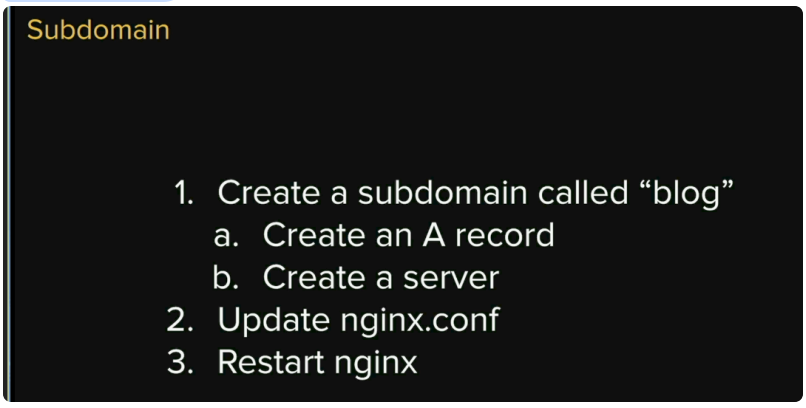



Đếm các bit rồi chuỗi nó lại

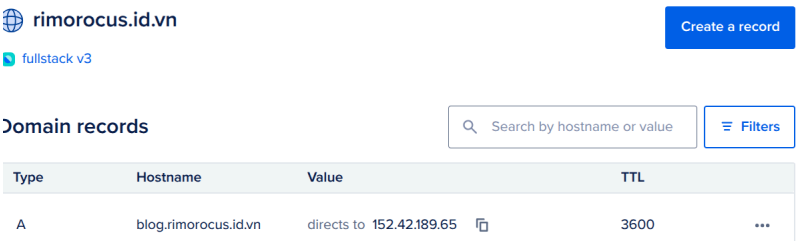
Compression khác với Hashing như thế nào?

- Compression: Mục tiêu giảm thiểu kích cỡ file
- Hashing: Mục tiêu để tạo các unique identifier

## Subdomain




Bước 1: Tạo A record cho subdomain



```
server {
        listen 80;
        listen [::]:80;
        server_name blog.<your_domain>;

        location / {
                proxy_pass http://localhost:3000;
        }
}
```

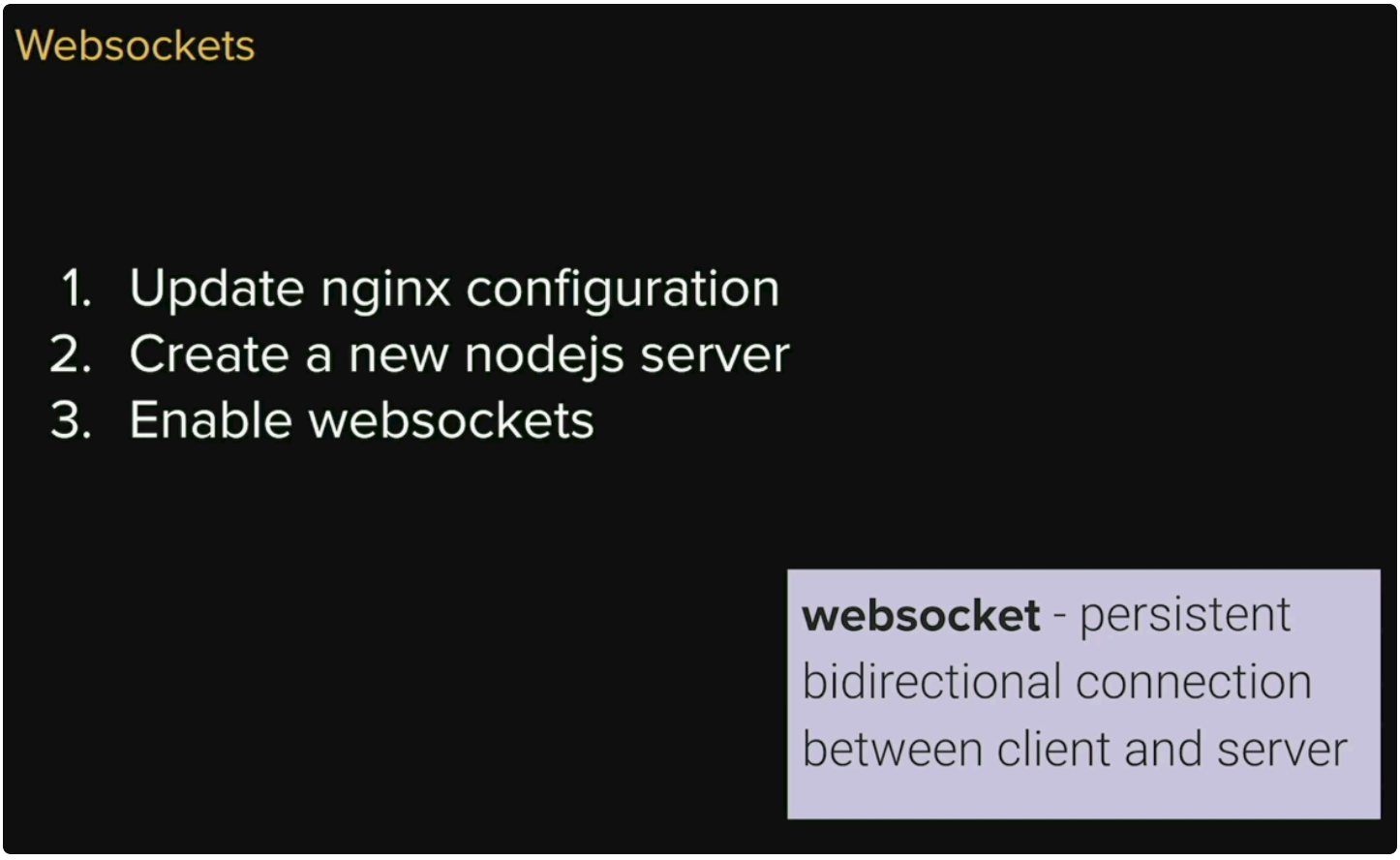
Tại sao lại phải dùng subdomain?
Để phát treeinr trong môi trường riêng biệt mà không phải tạo cookies

## Realtime & Databases

- Websockets
- Files
- Databases

Web socket

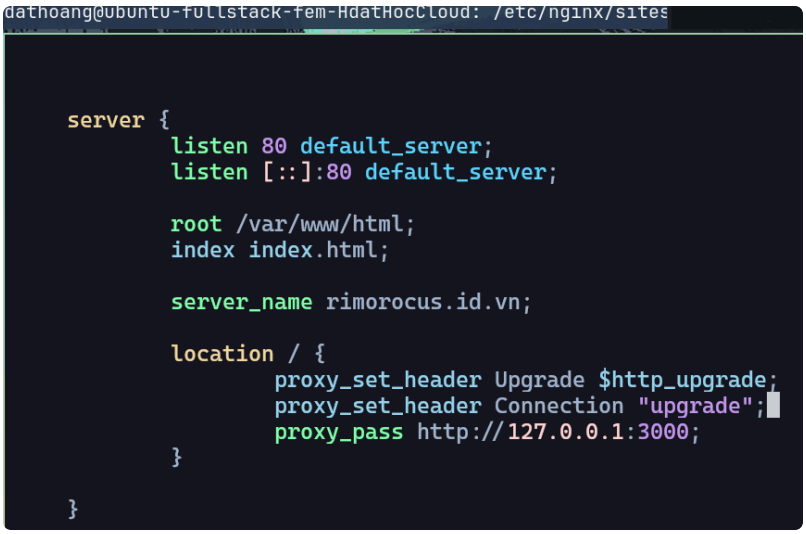
Là gì? Là một giao thức truyền thông để tạo ra **kết nối hai chiều liên tục** giữa client và server. Thực sự là realtime



| Đặc điểm | HTTP (Truyền thống) | WebSocket |
|---|---|---|
| **Cơ chế** | Request - Response (Hỏi - Đáp). | Full-duplex (Song công toàn phần). |
| **Chiều dữ liệu** | Client phải "hỏi" thì Server mới "trả lời". | Server có thể chủ động gửi dữ liệu cho Client mà không cần chờ hỏi. |
| **Kết nối** | Đóng lại ngay sau khi hoàn thành một yêu cầu. | Giữ kết nối mở liên tục cho đến khi một bên ngắt. |
| **Độ trễ** | Cao hơn (do phải thiết lập kết nối lại nhiều lần). | Thấp (thích hợp cho ứng dụng thời gian thực). |

Websocket được thiết kế để nằm trên TCP

thêm dòng này để config nginx cho websocket




Sử dụng Express (một web framework cho node.js)

## Websockets

```
const express = require('express');
const server = require('http').createServer();
const app = express();

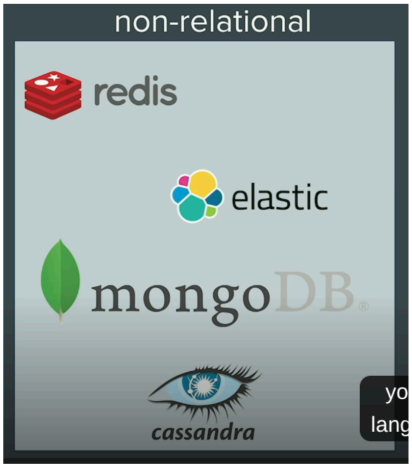
app.get('/', function(req, res) {
 res.sendFile('index.html', {root: __dirname});
});

server.on('request', app);
server.listen(3000, function () { console.log('Listening on 3000'); });
```

[fullstack-for-frontend3/apps/database/index-ws.js at main · young/fullstack-for-frontend3 · GitHub]

Databases




- sử dụng SQL tiêu chuẩn
- tuân thủ các luật nghiêm ngặt (ACID) (Atomicity, Consistency, Isolation, Durability)

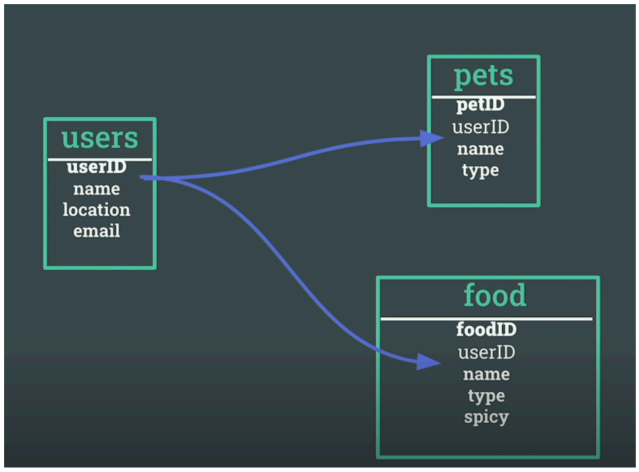
Khi nào dùng?
Khi ưu tiên TOÀN VẸN DỮ LIỆU và CẤU TRÚC

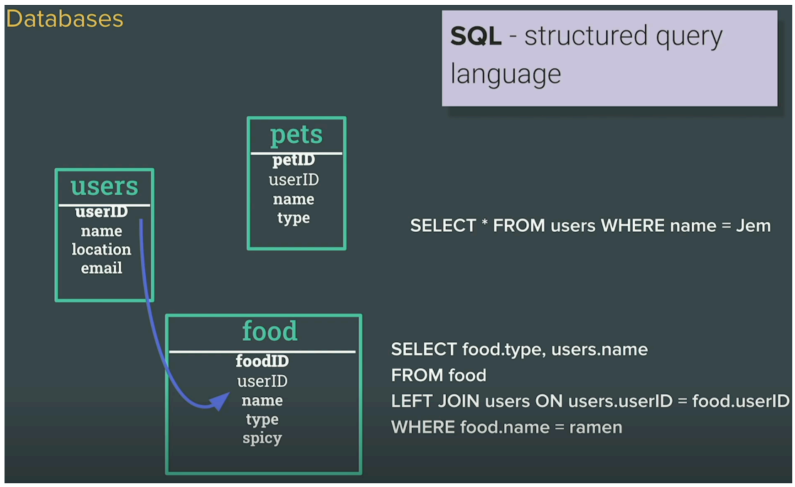
- Khi dữ liệu có cấu trúc ổn định
- Cần sự chính xác tuyệt đối
- Có quan hệ dlieu phức tạp
- Tuân thủ tiêu chuẩn

- sd cách truy vấn riêng
- cứ đổ lung tung hết vào cũng đc

Khi nào dùng?
Khi ưu tiên TỐC ĐỘ, LINH HOẠT và PHI CẤU TRÚC

- Dữ liệu k đồng nhất (mỗi file có attr khác nhau)
- Key-Value store: Cần đọc dlieu nhanh, để caching, quản lý phiên, tạm thời gì đó...
- Khi cần ghi liên tục dlieu siêu lớn (BigData), đẻ scale out dễ dàng (log hệ thống, IoT)
- Graph Database (Neo4j) dữ liệu chỉ là mqh giữa các entity




Dùng SQLite cho dự án này

